

Lab6 cow

Task

- 修改xv6代码，实现copy on write (COW)功能
 - 原本的fork：拷贝整个内存空间
 - Now：只记录映射，需要时才真的分配内存页
- 子进程在fork时通过页表share父进程的内存空间，所有**可写的页面**被设置为**只读页面**
- 任一进程在对**原本可写**的页面进行写操作时，触发page fault 并申请新的内存页面

Implementation

- 分配/释放内存页：现在一个页面可能同时被多个进程使用
 - 没有其他进程共享时才能释放
- 新数据结构 (kernel/kalloc.c) 管理每个页面的引用计数
 - 多进程并发：需要加锁
- 修改kalloc()/kfree()
 - kalloc(): 分配新页面时，初始化引用计数
 - kfree(): 减少计数，无其他人引用时释放

Implementation

- `uvmcopy( )` (`kernel/kalloc.c`) : fork时调用, 完成父进程内存空间复制
 - Now: 共享父进程的物理内存空间
- 删除申请新页面的逻辑, 只复制映射
- 检查pte、页面是否存在, 是否有效
- 识别权限: 只修改可写页面权限
 - 新权限位: COW (`kernel/riscv.h`)
 - 权限修改为只读+**COW**位
- 增加引用计数

Implementation

- `usertrap ( )` (`kernel/trap.c`): 在写COW页面时, 处理write缺页异常
 - 识别出该异常是因写了COW页面导致
 - 为原本的COW页面分配新页, 修改引用计数
- 识别COW异常
 - `r_scause()`读取异常原因: Store/AMO page fault, 15号
 - 对无权限的页面执行写操作
 - `r_stval()`得到异常地址, 进而得到需要修改的页面
 - 有效COW情况: **位于进程内存范围内(p->sz), !=0, 有效位, COW位**
- 处理异常:
 - 分配新页面, 复制页面内容
 - 修改权限位: 可写, 取消COW
 - `Kfree` 释放原pa

Implementation

- `copyout( )` (kernel/vm.c): 从内核复制数据到用户空间
- 类似`usertrap()`的处理
 - 判断是否是有效的COW页面
 - 拷贝内容
 - 修改权限位, `kfree`释放原地址

Tips

- 触发缺页异常时，可能是非法地址，而不是COW。需要排除
- COW权限位：区分原本只读/COW只读
- COW页错误需要分配内存，但空间不够：杀死进程
- Uvmcopy：检查PTE有效性！
- 创建新页面后：记得kfree！

Notice

- 在报告中附上make-grade的输出结果